

Atelier — Données nues puis objet

À ouvrir après 1-COURS.pdf du dossier projets/03-donnees-nues/.

1–3 périodes. Trois meules. Pas de billets de salle.

Le dossier depart/ de cet exercice contient les fichiers de départ. Vous n'avez pas à ouvrir un terminal.

Où l'on va

À la cave, vous savez décider si une meule est prête. Vous écrirez d'abord cette décision pour une ligne du tableau, sans classe. Avant de la recopier pour les deux autres lignes, vous rangerez la même règle dans `Meule.estPret()`.

À la fin, votre écran ressemble à ceci. Le tableau nu reste visible comme l'« avant » ; les trois cartes viennent ensuite des objets.

CAVE D'AFFINAGE Tableau nu Objets

SAS · FROMAGERIE · TROIS CAS

D'abord le tableau. Ensuite l'objet.

Une meule est prête si elle a au moins 60 jours d'affinage et au moins 4 kg. On voit la règle deux fois : en `if` dans le tableau, puis dans `Meule.estPret()`.

Les données nues

Aucune classe ici. Trois lignes, une décision écrite à la main.

Nom	Jours	Poids (kg)	Prête ?
Gruyère d'alpage	90	6	oui (90 ≥ 60 et 6 ≥ 4)
Tomme jeune	20	5	non (20 < 60)
Sérac léger	70	2	non (2 < 4)

Les mêmes trois, en objets

La page n'écrit plus le « oui / non ». Elle demande `estPret()` à chaque meule.

Gruyère d'alpage

AFFINAGE
90 jours

POIDS
6 kg

PRÊTE : OUI

Tomme jeune

AFFINAGE
20 jours

POIDS
5 kg

PRÊTE : NON

Sérac léger

AFFINAGE
70 jours

POIDS
2 kg

PRÊTE : NON

Sur un téléphone de 390 pixels de large, les cartes s'empilent et le tableau peut défiler horizontalement.

CAVE D’AFFINAGE

[Tableau nu](#)
[Objets](#)

SAS · FROMAGERIE · TROIS CAS

D’abord le tableau. Ensuite l’objet.

Une meule est prête si elle a au moins 60 jours d’affinage **et** au moins 4 kg. On voit la règle deux fois : en `if` dans le tableau, puis dans `Meule.estPret()`.

Les données nues

Aucune classe ici. Trois lignes, une décision écrite à la main.

Nom	Jours	Poids (kg)	Prête ?
Gruyère d’alpage	90	6	oui ($90 \geq 60$ et $6 \geq 4$)
Tomme jeune	20	5	non ($20 < 60$)
Sérac léger	70	2	non ($2 < 4$)

Les mêmes trois, en objets

Vous créez exactement ceci

```
sas/
├─ donnees-nues/
│  └─ index.html
│     └─ css/
│        └─ style.css
│           └─ js/
│              └─ meule.js
```

Le départ contient déjà le tableau dans `index.html`, le décor nuit/fromagerie dans `css/style.css` et le squelette de `Meule` dans `js/meule.js`.

Étape 1 — Lire le tableau sans classe

Vous avez déjà lu des données ligne par ligne : un nom, un nombre de jours et un poids. Pour l’instant, ces valeurs sont **nues** : aucun objet ne les rassemble et aucune méthode ne porte leur règle.

Placez le départ dans `sas/donnees-nues/`, puis ouvrez `sas/donnees-nues/index.html` par un double-clic. Vous devez voir trois lignes avec leur décision écrite dans le tableau et aucune carte sous le tableau.

- ☐ L'arbre correspond exactement au dessin ci-dessus.
- ☐ Gruyère d'alpage affiche 90 jours et 6 kg.
- ☐ Tomme jeune affiche 20 jours et 5 kg.
- ☐ Sérac léger affiche 70 jours et 2 kg.
- ☐ Les décisions sont écrites directement dans les trois lignes HTML.
- ☐ La zone « Les mêmes trois, en objets » est vide.

Le vide des cartes est normal. Vous n'avez à recopier ni le HTML ni le CSS depuis ce PDF.

Étape 2 — Décider pour une seule meule

Vous commencez comme dans un petit script sans objet. La règle nommée **meule prête** est : au moins 60 jours d'affinage **et** au moins 4 kg.

Dans `js/meule.js`, sous les trois constantes du Gruyère, écrivez ce premier `if` :

```
let verdictGruyere;  
if (joursGruyere >= 60 && poidsGruyere >= 4) {  
  verdictGruyere = "oui";  
} else {  
  verdictGruyere = "non";  
}  
console.log(verdictGruyere);
```

Rechargez, puis ouvrez la console. Le script doit répondre "oui", comme la première ligne du tableau.

Pour terminer sans classe, il faudrait recopier ce `if` pour la Tomme, puis encore pour le Sérac, en changeant les noms à chaque fois. La règle est courte, mais les deuxième et troisième copies créent déjà trois endroits à vérifier au prochain changement.

Étape 3 — Ranger la même règle dans l'objet

La classe `Meule` du départ est le moule. `new Meule(...)` produira un objet réel, comme un moule à billets produit un billet réel. Chaque objet connaîtra son nom, ses jours d'affinage et son poids.

Dans la méthode `estPret()`, remplacez le commentaire `/* à vous : une ligne */`. Gardez exactement la règle que vous venez d'écrire :

```
estPret() {  
  /* à vous : une ligne */
```

```
}
```

```
retourner vrai quand joursAffinage est au moins 60 ET poidsKg au moins 4
```

Ne laissez pas les nombres 90 et 6 dans la méthode. Elle doit fonctionner pour n'importe quel objet Meule grâce à `this.joursAffinage` et `this.poidsKg`.

Étape 4 — Fabriquer une seule meule

Avant de produire une liste, vérifiez le moule avec un objet précis. Sous la classe, fabriquez seulement le Gruyère :

```
const gruyere = new Meule("Gruyère d'alpage", 90, 6);
```

Rechargez, ouvrez la console et tapez :

```
gruyere.estPret();
```

Vous devez obtenir `true`. Le premier `if` et l'objet donnent la même réponse, mais la règle de l'objet est maintenant rangée à un seul endroit.

Étape 5 — Trois objets, trois cartes

Le test sur un objet fonctionne. Remplacez `const gruyere = ...` par les trois objets :

```
const cave = [
  new Meule("Gruyère d'alpage", 90, 6),
  new Meule("Tomme jeune", 20, 5),
  new Meule("Sérac léger", 70, 2),
];
```

Ajoutez ensuite l'affichage sous la liste. Il demande `estPret()` à chaque objet ; il ne recopie ni `>= 60` ni `>= 4` :

```
const liste = document.getElementById("liste");

cave.forEach((meule) => {
  const pret = meule.estPret();
  const article = document.createElement("article");
  article.className = "fiche";
  article.innerHTML = `
<h3>${meule.nom}</h3>
<dl>
<dt>Affinage</dt><dd>${meule.joursAffinage} jours</dd>
<dt>Poids</dt><dd>${meule.poidsKg} kg</dd>
</dl>
<p class="badge ${pret ? "badge--oui" : "badge--non"}">
Prête : ${pret ? "oui" : "non"}
```

```
</p>`;
liste.appendChild(article);
});
```

Rechargez. Le Gruyère répond « oui », la Tomme et le Sérac répondent « non ». Le tableau nu reste au-dessus : il montre le point de départ, tandis que les cartes interrogent les objets.

Réussi si

- ☐ Le tableau nu reste visible avec ses trois lignes.
- ☐ Votre premier `if` décide seulement pour le Gruyère.
- ☐ Vous savez expliquer pourquoi deux copies supplémentaires seraient pénibles.
- ☐ `gruyere.estPret()` répond `true` avant la création de la liste.
- ☐ Trois objets `Meule` existent à la fin.
- ☐ `estPret()` contient les deux conditions, une seule fois.
- ☐ La boucle demande `estPret()` aux objets sans recalculer la règle.
- ☐ La page se lit à 1440 et à 390 pixels de large.

AVEC L'IA

Vous avez écrit le premier `if` et `estPret()` vous-même. Montrez ces deux fragments à une IA et demandez : « Expliquez en quatre phrases ce qui a été déplacé, et ce qui n'a pas changé. » Sa réponse doit retrouver la même règle des 60 jours et des 4 kg.

POUR LES RAPIDES

Créez temporairement `new Meule("Tête de Moine", 80, 3.5)` dans la console et prédisez la réponse avant d'appeler `estPret()`. Modifiez ensuite une seule limite dans la méthode et observez quels objets changent de réponse ; remettez la limite d'origine avant de rendre.

Livrable

Rendez le dossier `sas/donnees-nues/` avec ses trois fichiers. La page doit s'ouvrir par double-clic et garder le tableau « avant ». À l'oral, vous montrez le premier `if`, puis l'endroit unique où la même règle vit dans `Meule.estPret()`.